

[Module 4] PETs for mobile systems

Q1: What are the pros and cons of the separation-of-duties concept? Please discuss from the perspectives of (computation, communication, and administrative) overhead, security and privacy.

Separation of Duties aims to separate who can do what, in order to protect anyone doing a mistake or hacker to hack the service. However it requires more computation because it needs to do more actions in order to be sure that it wasn't a mistake and to respect separation of duties; Separation of Duties (SoD) is designed to distribute responsibilities so that no single individual controls an entire process, thereby reducing the risk of errors, fraud, or malicious attacks. This approach helps protect against both accidental mistakes and intentional breaches by ensuring that critical actions require multiple approvals or oversight. However, implementing SoD introduces additional computation and administrative overhead. More steps and checks are needed to verify actions, maintain accountability, and enforce the separation, which can slow down processes and increase operational complexity. In financial transactions, separation of duties requires one employee to initiate a payment and another to approve it, reducing the risk of fraud or errors. However, this process adds extra steps and coordination, increasing both time and administrative effort.

Q2: Why does paper A choose to use group signatures? What does it achieve? Can we replace it with a traditional public key cryptographic algorithm (e.g., ECDSA)?

A group signature scheme is a method for allowing a member of a group to anonymously sign a message on behalf of the group. From that it is possible to know that a certain member of the group signed a message but it is not possible to tell who. It is interesting when you want to be sure that a message was really sent from someone who can send a message but it keeps anonymity. We can think about whistle-blower of a company, the whistle-blower might send a message signed with the company group signature, so it is certain that it is an employee of the company who sent the message but the employee would be fully anonymous.

It wouldn't be possible to do something like that with traditional key cryptography like RSA or ECDSA because each private key is assigned to someone and so if someone signs a message it's obvious from who the message was sent.

Q3: How does paper B protect querying nodes from the honest-but-curious peers? Please discuss from the perspectives of system policy and crypto usages.

This paper introduces solutions to safeguard user privacy in Location-Based Services (LBS) by shifting from centralized anonymizers to a decentralized, peer-to-peer (P2P) collaborative model. The core solution revolves around pseudonymous authentication, where users (nodes) obtain short-term, anonymous credentials from a Pseudonymous Certification Authority (PCA), ensuring their real identities remain hidden during interactions. To protect querying nodes, the system employs randomized serving node assignment, where a small, periodically rotated subset of nodes proactively caches and shares Point of Interest data for their region, reducing direct exposure to the LBS server. It is also possible to encrypt P2P communication to ensure that queries and responses are shielded from honest-but-curious peers, while cross-checking mechanisms allow nodes to validate the authenticity and accuracy of shared data by comparing responses from multiple serving nodes.

Q4: What are the pros and cons of centralized and decentralized approaches with respect to privacy (i.e., not design, we're talking about privacy guarantees).

With a centralized approach, the main server can see all data from anyone and so there is probably no anonymity, but it is easier to make an application run. Indeed with a central server, there is a backbone that runs the application and it's always accessible, with the same IP address, same protocol etc...

On the other hand, a decentralized approach enhances privacy because now only 2 peers that talk to each other in order to run the service know data about each other, but none of the two know everything about everyone. But there are drawbacks, it requires to have peers, and so people need to stay active even if they don't want to use that service.

Q5: What are the main challenges in doing proximity tracing between devices that cannot authenticate to each other and should not disclose their identity to anyone around?

If a user must stay anonymous and protect their identity the main problem for proximity tracing is that it is hard to know who is who and so track efficiently, for example if a vehicle enters a mixed-zone and before the location was known, now it is not possible anymore to track this vehicle. Furthermore another problem that comes with full-anonymity is that it is hard to counter Sibyl attacks or erroneous data submission, indeed if everyone is fully anonymous, it is impossible to know from which the packet was sent, and so impossible to have a revocation list.